ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. СЛОЖНОСТЬ АЛГОРИТМА, ЧАСТЬ 2

Время работы программы принято рассматривать как асимптотику числа её шагов в худшем случае. Но какое время считать хорошим, доступным для реализации на настоящих машинах? Точный ответ зависит от приложений. Так, программа, анализирующая граф всего интернета, может позволить себе фактически только линейное время работы. Возможно, подойдёт $O(n\log n)$, но никак не $O(n^2)$: такая программа будет попросту не успевать за изменением графа. С другой стороны, не слишком болышие системы линейных уравнений успешно решаются алгоритмами кубической сложности. Для общей теории хотелось бы иметь фундамент, не зависящий от деталей задачи и тонкостей реализации. В частности, класс быстрых алгоритмов не должен зависеть от точной спецификации вычислительной модели. В качестве такого класса был выбран класс алгоритмов с полиномиальным временем работы. Искомая независимость от модели достигается за счёт тезиса Кобхэма–Эдмондса: при переходе к друтой модели время работы может увеличиться в полиномиальное число раз, но полином от полинома остаётся полиномом. Кроме того, желательно, чтобы композиция эффективных алгоритмов была эффективным алгоритмом. Класс полиномов - минимальный класс, содержащий линейные функции и замкнутый относительно композиции. Конечно, алгоритм со временем работы $O(n^{100})$ или с очень болышой константой в $O(\cdot)$ – обозначении на практике не реализуем, но настолько больши́е степени полинома крайне редко встречаются в реальных алгоритмах. Более того, если открыт алгоритм с большим полиномиальным временем работы, то обычно в последующих работах степень уменьшается до приемлемых значений. (Например, знаменитый AKS-алгоритм проверки простоты изначально имел сложность $\tilde{O}(n^{12})$ для n-значного числа, а затем был улучшен до $\bar{O}(n^6)$). Однако есть и следующие ступени (не)доступности. Так, для нескольких задач неизвестно полиномиального алгоритма, но известен квазиполиномиальный, работающий за время $O\left(n^{\text{polylog } n}\right)$. В этой связи стоит отметить недавнее достижение Ласло Бабаи, построившего квазиполиномиальный алгоритм для проверки изоморфности двух графов.

4 января 2017 года Бабаи отозввл своё утверждение о кввзиполиномнальности, заменив на более скромную субэкспоненциальную оценку. Однако уже 9 января квазинолиномиальность была восстановлена.

1.1. Стандартные классы сложности

Классом **P** называется множество языков, распознающихся за полиномиальное время. Формально он определяется так:

$$\boldsymbol{P} = \bigcup_{c=1}^{\infty} \mathrm{DTIME}(n^c).$$

Можно упрощённо записать это определение как $\mathbf{P} = \mathrm{DTIME}(\mathrm{poly}(n))$. Примерами языков из $\mathbf{P}$ являются такие множества:

$\mathrm{GCD} = \{(a, b, d) \mid$ число d является наибольшшм общим делителем чисел a и $b\}$. Действительно, наибольший общий делитель можно найти алторитмом Евклида и затем сравнить резултат с d.

$\mathrm{PATH} = \{(G, s, t) \mid$ в графе G есть путь из s в $t\}$. Наличие пути в графе проверяется быстро, например обходом в ширину.

$\mathrm{CONNECTED} = \{G \mid G$ - связный граф$\}$. Например, можно проверить граф на связность, проверив наличие пути из некоторой фиксированной вершины во все остальные, или использовать обход графа единовременно.

$\mathrm{EULERCYCLE} = \{G \mid$ в графе G есть эйлеров цикл $\}$. По критерию существования эйлерова цикла достаточно проверить связность, а также посчитать степени всех вершин и проверить, что все они чётные.

$\mathrm{BIPARTITE} = \{G \mid$ граф G двудольный $\}$. Иными словами, вершины графа G можно покрасить в 2 цвета, так чтобы вершины одного цвета не были соединены ребром. В данном случае подойдёт «жадный» алгоритм: покрасим одну вершину произвольно, дальше будем красить соседние с уже покрашенными согласно условию, пока не придём к противоречию или не покрасим всю компоненту связности. В последнем случае продолжим тем же алгоритмом, пока все компоненты не кончатся. Если в какой-то момент случилось противоречие, значит граф содержит нечётный цикл, а тогда он не является двудольным.

$\mathrm{MINCUT} = \{(G, k) \mid$ вершины графа G можно разбить на два множества S и T, так что между частями проведено не более k рёбер $\}$. Эта задача решается широко известным алгоритмом Форда-Фалкерсона с исполззованием теоремы о максимальном потоке и минимальном разрезе.

Обратите внимание, что если в постановку задачи входит числовой параметр n, то полином считается не от этого параметра, а от его логарифма, т.к. на запись числа n требуется $\log n$ битов. Если алоритм работает полиномиальное время от самого n, то такой алоритм называется псевдополинамиальныи. Существуют задачи, для которых не известно полиномиального алторитма, но известен псевдополиномиальный. Например, такими задачами являются задача о рюкзаке

$$\mathrm{KNAPSACK} = \left\{(n_1, \dots, n_k, m_1, \dots, m_k, N, M) \mid \exists \alpha \in \{0,1\}^k \sum \alpha_i n_i \leqslant N \text{ и } \sum \alpha_i m_i \geqslant M\right\},$$

и её частный случай

$$\text{SUBSET-SUM} = \left\{(n_1, \dots, n_k, N) \mid \exists \alpha \in \{0,1\}^k \sum \alpha_i n_i = N\right\}.$$

Интерпретация задачи о рюкзаке такая: имеются предметы с весами n_i и стоимостями m_i. Вопрос: можно ли взять в рюкзак несколько предметов, так чтобы их суммарный вес не превысил N (иначе рюкзак порвётся), а суммарная стоимость

была не меньше M. В задаче SUBSET-SUM остаётся один параметр: ставится вопрос о том, можно ли заполнить рюкзак, не оставляя свободного места.

Нетрудно заметить следующие соотношения для класса **P** :

Теорема. Если $L \in \mathbf{P}$ и $M \in \mathbf{P}$, mo $\bar{L}, L \cup M, L \cap M$ также лежат в **P**.

Иначе говоря, класс **P** замкнут относнтельно операций дополнения, объединения, пересечения.

Доказательство. Действительно, имея ответ на вопрос, принадлежит ли слово x языкам L и M, простейшими логическими операциями (отрицание, дизъюнкция, конъюнкция) можно получить ответы на вопросы о принадлежности слова x языкам $\bar{L}, L \cup M, L \cap M$ соответственно. Эти операции займут совсем немного времени, поэтому время работы останется полиномиальным. Более точно, если $L \in \mathrm{DTIME}(T(n))$, то $\bar{L} \in \mathrm{DTIME}(T(n))$, а если $L \in \mathrm{DTIME}(T(n))$ и $M \in \mathrm{DTIME}(S(n))$, то $L \cup M$ и $L \cap M$ лежат в DTIME $(\max\{T(n), S(n)\})$.

Помимо класса **P** изучают более высокие классы, определяемые порядком роста времени работы на машине Тьюринга:

$\mathrm{QP} = \mathrm{DTIME}\left(2^{\mathrm{polylog}(n)}\right) = \bigcup_{c=1}^{\infty} \mathrm{DTIME}\left(2^{\log^c n}\right)$ (квазиполиномиальное время);

$\mathrm{SUBEXP} = \mathrm{DTIME}\left(2^{n^{\mathrm{o}(1)}}\right) = \bigcap_{e>0} \mathrm{DTIME}\left(2^{n^{\varepsilon}}\right)$ (субэкспоненциальное время);

$\mathrm{E} = \mathrm{DTIME}\left(2^{\circ(n)}\right) = \bigcup_{c=1}^{\infty} \mathrm{DTIME}(2^{\infty n})$ (линейно-экспоненциальное вреМЯ) ;

$\mathbf{EXP} = \mathrm{DTIME}\left(2^{\mathrm{poly(n)}}\right) = \bigcup_{c=1}^{\infty} \mathrm{DTIME}\left(2^{n^c}\right)$ (экспоненциальное время, иногда его также называют EXPTIME);

$\mathbf{EE} = \mathrm{DTIME}\left(2^{2^{2(n)}}\right) = \bigcup_{c=1}^{\infty} \mathrm{DTIME}\left(2^{2-\infty}\right)$ (дважды линейно-экспоненциальное время);

$\mathrm{EEXP} = \mathrm{DTIME}\left(2^{2^{\mathrm{poly(n)}}}\right) = \bigcup_{c=1}^{\infty} \mathrm{DTIME}\left(2^{2^{n^0}}\right)$ (дважды экспоненциальное время);

и т.д.

Очевидно, что $\mathrm{P} \subset \mathrm{QP} \subset \mathrm{SUBEXP} \subset \mathrm{E} \subset \mathrm{EXP} \subset \mathrm{EE} \subset \mathrm{EEXP}$. Из теоремы об нерархии следует, что все эти вложения строгие. Доказательство этой теоремы мы пока отложим. Стоит также обратить внимание на то, что SUBEXP в отличие от остальных классов определяется не через объединение, а через пересечение различных классов. Это приводит к потенциальной неоднородности: может не быть единого алгоритма, время работы которого растёт медленнее любой функции $2^{n^{\varepsilon}}$, но для каждого ε найдётся свой такой алгоритм.

Как правило, включение задачи в тот или иной класс, например, **P**, доказывается предъявлением алгоритма соответствующей сложности. Может показаться, что никакого другого способа и быть не может. Тем удивительнее, что бывает иначе. Есть целый класс задач, лежащих в P, для которых неясно, как построить конкретный алгоритм! Они связаны с топологическими свойствами графов.

Известен критерий Понтрягина–Куратовского планарности графов: граф планерен тогда и только тогда, когда в нём нет подграфа, гомеоморфного K_5 или $K_{3,3}$ (т.е. полному подграфу из 5 вершин или полному двудольному подграфу с двумя долями по 3 вершины). Наличие таких подграфов проверяется полиномиальным перебором: например, для K_5 нужно перебрать все наборы из 5 вершин и проверить наличие всех путей между всеми 10 парами. Алгоритм будет полиномиален, поскольку нужно перебрать $C_n^5 = O(n^5)$ вариантов, и проверка на наличие пути также полиномиальна. Ясно, что аналогично можно проверить вложенность любого конкретного подграфа. Заметим, что для проверки планарности есть и более эффективные алгоритмы, например, линейный алгоритм Хопкрофта-Тарджана.

Планарность графа не теряется, если из него удалить ребро или если ребро стянуть. На этом основан критерий планарности Вагнера: граф планерен тогда и только тогда, когда в нём нет миноров K_5 или $K_{3,3}$. Минором как раз называется граф, полученный последовательностью удалений и стягиваний рёбер. Легко понять, что если у графа есть подграф, гомеоморфный H, то у него есть и минор H. Обратное не всегда верно, тем не менее в критерии Вагнера и критерии Понтрагина-Куратовского набор запрещённых графов одинаков.

Многие другие топологические свойства также сохраняются при переходе к минорам: вложимости графа без самопересечений на какую-нибудь двумерную поверхность, размещение его в пространстве, так чтобы все циклы были не заузлены и/или не зацеплены друг с другом, и т.д. Гипотеза Вагнера заключалась в том, что для любого такого свойства существует конечное число запрещённых миноров. В огромной серии из 23 статей, выходившей с 1983 по 2012 годы Нил Робертсон и Пол Сеймур разработали теорию графовых миноров, где среди прочего построил полиномиальный алгоритм проверки, есть ли в графе данный минор и доказали гипотезу Вагнера. Тем самым, для всех таких топологических свойств существуют полиномиальные алгоритмы. Вот только теорема не даёт явного списка запрещённых миноров, и потому непонятно, как именно устроен алгоритм и какая там степень полинома. Даже для вложимости в тор полного списка запрещённых миноров пока нет.

В 2013 году Берт Джерардз, Джим Джилин и Джефф Уиттл заявили, что доказали аналогичную теорему в более общем случае, а именно для замкнутых относительно взятия миноров классов матроидов, представимых над конечным полем. Здесь также есть лишь конечное число запрещённых миноров, и вхождение каждого из них проверяется за полиномиальное время. Стоит отметить, что теорема о конечном числе запрещённых миноров есть и для самого свойства представимости над данным полем - в этом заключалась гипотеза Роты. Но в этом случае нет теоремы о полиномиальном алгоритме; более того, при некоторой спецификации модели его точно нет.

Недетерминированные вычисления – важная теоретическая концепция, хотя и не реализуемая в виде физического устройства.

Некоторые полагают, что недетерминизм можно моделировать на квантовом компьютере. Это распространённое заблуждение, сродни тому что «в геометрии Лобачевского параллельные прямые пересекаются». К сожалению, этому заблуждению порой подвержены и серьёзные учёные. Так, Дэвид Дойч ошибочно использовал такой довод в пользу своей теории множественности миров: «Если параллельных миров нет, то откуда же квантовые компьютеры черпают свою силу?» Здесь подразумевается, что в параллельных мирах производятся экспоненциальный перебор, но квантовые компьютеры устроены не так и проводить перебор не умеют.

Неформально говоря, недетерминизм позволяет проводить перебор экспоненциальных семейств слов за один шаг, сразу угадывая нужное слово. Формально недетерминированные вычисления проводятся на недетерминированной машине Тьюринга.

Недетерминированной маииной Тьюрига с k лентами называется кортеж $\langle \Sigma, \Gamma, Q, q_0, q_a, q_r, k, \delta \rangle$, где Σ, Γ и Q суть конечные непустые множества, причём $\Sigma \subset \Gamma$ и $\Gamma \cap Q = \emptyset, q_0, q_a, q_r \in Q$ и попарно различны, а $\delta: (Q \setminus \{q_a, q_r\}) \times \Gamma^k \rightrightarrows Q \times \Gamma^k \times \{L, N, R\}^k$ – всюду определённая многозначная функция.

Таким образом, единственным отличием от детерминированной машины является многозначность функции перехода δ. Гораздо сильнее отличие в определении того, что такое вычисление и что означает распознавание языка. Суть состоит в том, что из каждой конфигурации машина может перейти не в какую-то одну следующую, а в любую совместимую с функцией перехода. Формально это определяется так:

Пусть $C = (a_1\sigma_1, \dots, a_k\sigma_k; \tau_1 b_1, \dots, \tau_k b_k; q)$ и $C' = (a'_1, \dots, a'_k; b'_1, \dots, b'_k; r)$ – некоторые конфигурации, где $q \neq q_a$ и $q \neq q_r$. Тогда переход от C к C' допустиім, если среди значений $\delta(q, \tau_1, \dots, \tau_k)$ найдётся $(r, \rho_1, \dots, \rho_k, D_1, \dots, D_k)$, такое что выполнено одно из трёх:

$$D_i = L, a'_i = a_i \ (\text{или } a'_i = \#, \text{если } a_i = \varepsilon) \text{ и } b'_i = \sigma_i \rho_i b_i;$$

$$D_i = N, a'_i = a_i \sigma_i \text{ и } b'_i = \rho_i b_i;$$

$$D_i = R, a'_i = a_i \sigma_1 \rho_i \text{ и } b'_i = b_i \ (\text{или } b'_i = \#, \text{если } b_i = \varepsilon).$$

Не следует думать, что машина выбирает следующую конфигурацию случайно. Лучше представлять, что машина каким-то образом разделяется на несколько копий, каждая из которых выбирает свой переход. На следующем шаге каждая из копий ещё подразделяется, и так далее. В результате каждая копия производит своё вычисление.

Цепочка конфигураций $(C_1, \dots, C_n)$ называется вычислением, если все переходы от C_i к C_{i+1} допустимы.

Удобно считать, что все вычисления собираются в одно укоренённое дерево: корнем будет начальная конфигурация, а непосредственными потомками каждой вершины - те конфигурации, переходы в которые допустимы. Как обычно, будем

считать, что машина, получившая на вход слово x, начинает вычисление в конфигурации $(\underbrace{\#,\dots,\#}_{k\text{ раз}};x,\underbrace{\#\dots,\#}_{k-1\text{ раз}};q_0)$. Ниже мы будем просто писать, что вычисление начинается с x. Теперь надо определить, как машина собирает исходы всех вычислений в один результат своей работы. Неформально говоря, если одна из копий пришла в принимающая состояние, то она сразу сигнализирует об этом в управляющий центр. Если центр получил хотя бы один такой сигнал за отведённое время, то он принимает вход, иначе отвергает. Формально определение следующее.

Недетерминированная машина M распознаёт язык L за время $T(n)$, если при всех $x \in \{0,1\}^*$ верно следующее:

любое вычисление, начинающееся с x, не длиннее $T(|x|)$ (иначе говоря, машина в любом случае останавливается не более чем за $T(|x|)$ шагов);

если $x \in L$, то найдётся вычисление, начинающееся с x и заканчивающееся в состоянии q_a;

если $x \notin L$, то любое вычисление, начинающееся с x, заканчивается в состоянии q_r.

Недетерминированные вычисления позволяют определить соответствующий сложностной класс.

Классом $\mathrm{NTIME}(T(n))$ называется множество языков, распознаваемых на недетерминированной машине Тьюринга за время $O(T(n))$.

Некоторые авторы говорят не о многозначной, а о двузначной функции перехода δ, или о двух функциях δ_0 и δ_1. От такой вариации время работы изменится в константу раз, поскольку множество значений функции δ имеет константный (т.е. зависящий не от длины входа, а только от параметров машины) размер, поэтому выбор нужного значения можно смоделировать как константное число последовательных бинарных выборов. Точный размер множества значений – $|Q| \cdot |\Gamma|^k \cdot 3^k$.

Легко установить следующую связь между детерминированными и недетерминированными вычислениями.

Теорема. $\mathrm{DTIME}(T(n)) \subset \mathrm{NTIME}(T(n)) \subset \mathrm{DTIME}(2^{O(T(n))})$.

Как и раньше, запись в правой части толкуется как $\bigcup_{c=1}^{\infty}\mathrm{DTIME}(2^{cT(n)})$. В дальнейшем мы также будем пользоваться подобными сокращениями.

Доказательство. Первое вложение следует из того, что обычная машина Тьюринга – частный случай недетерминированной. Для второго вложения вспомним, что у функции перехода может быть не больше некоторой константы K значений. Значит, общее количество конфигураций во всех вычислениях, начинающихся с данного входа и имеющих длину не больше T, не превышает $K^T = 2^{O(T)}$. За такое же время можно все эти конфигурации перебрать (например, обходя дерево) и проверить, встречается ли среди них состояние q_a. Если встречается, то данный вход лежит в языке, иначе не лежит.

Класс NP важен как для теории, так и для практики. У него есть два эквивалентных определения: через недетерминированные машины и через сертификаты.

$\mathrm{NP} = \bigcup_{c=1}^{\infty} \mathrm{NTIME}(n^c)$.

Это определение объясняет название NP: N – недетерминированные вычисления, P – полиномиальное время. Тут тоже встречается популярная ошибка; люди считают, что раз **P** означвет полиномиальное время, то NP – не полиномнальное. Это, конечно, не так, ведь $\mathbf{P} \subset \mathbf{NP}$.

Классом NP называется множество языков L, для которых существует функция $V(x, s)$ с булевыми значениями, вычислимая за полиномиальное время от длины первого аргумента, такая что:

если $x \in L$, то $\exists s \;\; V(x, s) = 1$;

если $x \notin L$, то $\forall s \;\; V(x, s) = 0$.

Второй вход s часто называют сертификатам, а функцию V – верификатором. Таким образом, сертификат удостоверяет, что $x \in L$, а верификатор проверяет верность сертификата. Для слов из языка подходящий сертификат должен существовать, а для слов не из языка все сертификаты должны отвергаться. Иначе говоря, класс NP – это класс языков, принадлежность к которым можно быстро доказать, а **P** – класс языков, принадлежность к которым можно быстро выяснить.

Оговорка о том, что время работы V полиномиально именно от длины x, важна, иначе в сертификат можно было бы включить слишком много информации и сделать любое время работы полиномиальным. Альтернативно можно потребовать, чтобы алгоритм V был полиномиальным от своего входа целиком, но вот длина сертификата была бы ограничена полиномом от длины x.

Теорема. Определения класса NP через недетерминированные машины и через сертификаты эквиваленты, т.е. задают один и тот же класс языков.

Доказательство. Пусть L распознаётся недетерминированной машиной M, которая работает $O(n^c)$ шагов. Тогда в качестве сертификата можно взять последовательность значений функции перехода, а верификатор будет моделировать работу машины M, используя данные сертификата для выбора одной из ветвей алгоритма. Можно действовать и по-другому: в качестве сертификата взять всё вычисление, а верификатором проверать его соответствие программе.

Пусть, напротив, для L верно сертификатное определение. Тогда сертификат не может быть длиннее, чем время работы V, т.е. чем некоторый полином $p(|x|)$. Недетерминированная машина будет работать следующим образом: сначала она недетерминированно напишет сертификат 8 длины не больше $p(|x|)$, а затем запустит $V(x, s)$.

Примерами языков из NP служат такие задачи:

$\mathrm{SAT} = \{\varphi \mid \varphi -$ выполнимая булева формула$\}$. (Выполнимость означает, что формула равна 1 на некотором наборе значений). В данном случае сертификатом

будет выполняющий набор, а верификатор проверит, что значение формулы на этом наборе действительно равно 1. Важным частным случаем является задача 3SAT, в которой про формулу φ дополнительно сказано, что она представлена в виде 3-КНФ, т.е. КНФ, в которой каждый дизъюнкт включает в себя 3 литерала.

CLIQUE $= \{(G, k) \mid$ в графе G найдётся полный подграф хотя бы на k вершинах $\}$. Сертификатом будет сама клика.

3COL $= \{G \mid$ вершины графа G можно правильно раскрасить в 3 цвета $\}$. Сертификатом будет раскраска.

HAMCYCLE $= \{G \mid$ в графе G существует гамильтонов цикл $\}$. (Гамильтонов цикл проходит ровно 1 раз через каждую вершину). Сертификатом будет сам цикл.

KNAPSACK, SUBSET-SUM (см. выше). Сертификатом будет набор α.

FACTORING $= \{(N, a, b) \mid$ у числа N существует простой делитель на отрезке $[a, b]\}$. Сертификатом будет этот делитель.

MAXCUT $= \{(G, k) \mid$ вершины графа G можно разбить на два множества S и T, так что между частями проведено не менее k рёбер$\}$. Сертификатом будет это разбиение.

Задачи из класса NP встречаются очень часто в дискретной математике, алгебре, логике, дискретной оптимизации и других разделах конечной математики.

Как и класс **P**, класс NP замкнут относительно объединения и пересечения языков (но не дополнения!). Более того, можно доказать такую общую теорему.

Теорема. Если $L \in \mathrm{NTIME}(T(n))$, а $M \in \mathrm{NTIME}(S(n))$, то $L \cup M$ и $L \cap M$ лежит в $\mathrm{NTIME}(\max\{T(n), S(n)\})$.

Доказательство. Недетерминированная машина должна сначала провести вычисление для L, затем для M, а затем взять дизъюнкцию или конъюнкцию результатов для $L \cup M$ и $L \cap M$ соответственно. В случае с дизъюнкцией дтя элементов $L \cup M$ хотя бы на одной ветви хотя бы одного из этапов ответ будет положительным, поэтому на соответствующей ветви общего алгоритма ответ будет положительный, а для слов не из $L \cup M$ на всех ветвях обоих этапов будет отрицательный ответ, поэтому и итоговый ответ будет отрицательным. В случае с конъюнкцией для элементов $L \cap M$ на какой-то ветви на каждом этапе ответ будет положительным, поэтому на составной ветви ответ также будет положительным. Для слов не из $L \cap M$ на каком-то из этапов ответы на всех ветвях будут отрицательными, поэтому и итоговый ответ будет отрицательным.

Время работы будет примерно равно сумме $O(T(n))$ и $O(S(n))$, что также представимо в виде $O(\max\{T(n), S(n)\})$.

Проблема перебора ($\mathbf{P} \stackrel{?}{=} \mathrm{NP}$).

Мы уже выяснили, что $\mathrm{DTIME}\big(T(n)\big) \subset \mathrm{NTIME}\big(T(n)\big)$, откуда $\mathbf{P} \subset \mathrm{NP}$. Также $\mathbf{NP} \subset \mathbf{EXP}$. Вскоре мы докажем, что хотя бы одно из этих вложений строгое, т.к. $\mathbf{P} \neq \mathrm{EXP}$. Однако вопрос о том, являюття ли строгими оба вложения, открыт. Наибольшее внимание привлекает вопрос о равенстве классов P и NP. Он также называется

проблемой перебора, потому что может быть сформулирован так: возможно ли смоделировать экспоненциальный перебор возможных сертификатов за полиномиальное время? Существенная часть теории сложности вычислений и подавляющая часть теоретической криштографии разработаны в предположении $\mathrm{P} \neq \mathrm{NP}$. Однако это не единственное недоказанное предположение. Существуют и другие: неколлапсирование полиномиальной иерархии, существование односторонней функции и др. Исходный вопрос поставлен в начале 1970-х Стивеном Куком и Леонидом Левиным, однако до сих пор все существенные продвижения лишь дополнительно показывали, что вопрос сложный:

В оригинальных работах Кук и Левин независимо друг от друга открыли класс NP-полных задач, или универсальных задач перебора, «самых сложных в классе NP. В настоящее время известны тысячи таких задач, с вариациями – десятки тысяч. Нахождение полиномиального алгоритма для любой из них привело бы к тому, что $\mathrm{P} = \mathrm{NP}$, однако многолетние усилия огромного числа учёных по поиску такого алгоритма успехом не увенчались.

В 1975 году Бейкер, Джилл и Соловей доказали, что проблема $\mathbf{P} \stackrel{?}{=} \mathrm{NP}$ не релятивизуется, т.е. существует оракул A, такой что $\mathbf{P}^A = \mathbf{NP}^A$, и существует оракул B, такой что $\mathbf{P}^B \neq \mathrm{NP}^B$. Это значит, что любое правильное доказательство должно быть нерелятивизуемым, а примеров таких доказательств не так много.

В 1994 году Разборов и Рудич ввели понятие естественного доказательства и доказали следующий факт: если верно некоторое усиление $\mathbf{P} \neq \mathrm{NP}$, то само утверждение $\mathbf{P} \neq \mathbf{NP}$ невозможно доказать при помощи естественных доказательств. На совсем неформальном уровне идея такова: утверждение $\mathbf{P} \neq \mathrm{NP}$ говорит как раз о том, что поиск доказательства гораздо сложнее, чем его проверка. Но это соображение можно применить и к самому утверждению $\mathbf{P} \neq \mathbf{NP}$! Именно поэтому мы не можем найти доказательства этого утверждения. Разборов и Рудич уточняют эту идею: если некоторая техника позволяет доказать, что некоторая задача сложная, то эту же технику можно развернуть для доказательства, что некоторая другая задача простая. А простота этой задачи и будет противоречить упомянутому усилению $\mathbf{P} \neq \mathbf{NP}$.

В 2008 году Ааронсон и Вигдерсон доказали, что гипотетическое доказательство $\mathbf{P} \neq \mathrm{NP}$ не может использовать салгебраические техники», что дополнительно исключило многие известные методы. Формально они доказали аналог результата Бейкера–Джилла–Соловея для оракулов специального вида, названных алгебраическими.

Помимо вариантов совпадения и несовпадения классов $\mathbf{P}$ и $\mathbf{NP}$ есть и экзотический вариант: утверждение $\mathbf{P} = \mathrm{NP}$ не зависит от остальных аксиом, подобно аксиоме выбора или континуум-гипотезе. С одной стороны, имеется несколько результатов, показывающих независимость утверждения $\mathbf{P} = \mathbf{NP}$ от некоторых слабых аксиоматических теорий. С другой стороны, показано, что если

$\mathrm{P} = \mathrm{NP}$ не зависит от арифметики Пеано, то **P** и NP «почти равны». А именно, NP вложено в $\mathrm{DTIME}\left(n^{\log^* n}\right)$. (Функция $\log^* n$ равна количеству итераций логарифмирования, которое нужно применить к n, чтобы получить число меньше 2. Для всех реальных значений n эта функция не превосходит 5 , хотя асимштотически и стремится к бесконечности.) Теоретически возможны и другие странные» варианты, связанные с тем, что $\mathrm{NP} \subset \mathrm{SUBEXP}$.

При этом из-за того, что за решение проблемы перебора Институтом Клэя назначена премия в миллион долларов, вопрос привлекает внимание многих непрофессионалов, в результате несколько раз в год появляются заявления о том, что проблема решена в ту или иную сторону. Как правило, найти ошибку в рассуждениях не составляет большого труда, но случаются и более изощрённые попытки, например, работа Виная Деолаликара 2010 года, вызвавшая переполох в среде теоретиков. Сообщество быстро самоорганизовалось, пристально изучило стостраничную статью и нашло «дырку», которая пока не заделана.

Также многие обыватели, завороженные словосочетанием «квантовый компьютер», ошибочно полагают, что изобретение квантового компьютера позволит решать все задачи из NP за полиномиальное время. Однако на современном уровне знаний это не так: про задачу о разложении числа на множители (которую быстро решает квантовый компьютер) не доказана NP-полнота, а для NP-полных задач не известно квантовых алгоритмов.

Вопрос о равенстве **P** и NP настолько важен, что в 1989 году его увековечили в камне на стене факультета компьютерных наук принстонского университета (рис. 56). Углубления в стене можно прочесть как $\mathrm{P} = \mathrm{NP}?$ в 7-битной ASCII-кодировке. Такая кодировка давно не используется, а теоретический вопрос всё так же стоит.



Рисунок 1. Кирпичная кладка на стене факультета компьютерных наук принстонского университета. Каждый отсутствующий кирпич обозначает единицу, присутствующий – ноль. Строки кодируют в семибитной ASCII-записи вопрос $P = NP$?

В отличие от класса $\mathbf{P}$, для класса NP скорее всего не верна замкнутость относительно дополнения. Действительно, непонятно, как можно быстро доказать отсутствие подходящего сертификата. Дополнения языков из NP объединяют в отдельный класс coNP.

$\text{coNP} = \{L \mid \bar{L} \in \mathbf{NP}\}$.

Иначе говоря, если к NP относятся языки, принадлежность к которым легко доказать, то coNP – класс языков, принадлежность к которым легко опровергнуть. Самым естественным представителем класса coNP является язык тавтологий $\text{TAUT} = \{\varphi \mid \varphi - \text{ тавтология}\}$. Действительно, опровергнуть, что φ тавтология, очень легко: нужно предъявить набор значений, на котором формула ложна. Доказать, что φ тавтология, можно, предъявив вывод в исчислении высказываний, однако такой вывод может быть слишком длинным: длиннее, чем полином от длины φ. Коротких доказательств тавтологичности не известно.

Легко показать, что $\mathbf{P} \subset \mathbf{coNP} \subset \mathbf{EXP}$. Таким образом, $\mathbf{P} \subset \mathbf{NP} \cap \mathbf{coNP}$. Про это вложение тоже неизвестно, строго ли оно, однако большинство исследователей верят, что строго. В частности, задача FACTORING лежит в NP $\cap$ coNP: доказать наличие делителя в нужном интервале можно, просто предъявив его, а опровергнуть - приведя полное разложение на простые множители. Имея полиномиальный алгоритм проверки простоты, это разложение можно проверить и таким образом убедиться в отсутствии простых множителей в заданном интервале.

Впрочем, можно обойтись и без полиномиального теста простоты. Достаточно, что само множество простых чисел лежит в NP $\cap$ coNP. Принадлежность coNP очевидна: для составного числа предъявляется разложение на множители, – а простота удостоверяется так: n простое тогда и только тогда, когда мультипликативная группа Z_n^* циклическая. В сертификат нужно включить генератор g этой группы и доказательство ее цикличности. Это доказательство заключается в предъявлении разложения $n-1$ на простые множители p_i тогда можно проверить $g^{n/p_i} \neq 1$. А простота этих множителей удостоверяется подобными же сертификатами индуктивно. Также индуктивно доказывается, что общая длина всех сертификатов полиномиальна.

При этом научиться решать FACTORING достаточно для последующего поиска разложения на множители, достаточно воспользоваться двоичным поиском. А эта задача, по всей видимости, не решается за полиномиальное время.

Также легко показать такую связь.

Теорема. Если $\mathbf{P} = \mathbf{NP}, m_o\mathbf{NP} = \text{coNP} = \mathbf{P}$.

Доказательство. Действительно, пусть $L \in \text{coNP}$. Значит, $\bar{L} \in \mathbf{NP}$. Поскольку мы предположили, что $\mathbf{P} = \mathbf{NP}$, то $\bar{L} \in \mathbf{P}$. Поскольку классе $\mathbf{P}$ замкнут относительно

дополнения, то и $L \in \mathbf{P}$. Значит, coNP $\subset \mathbf{P}$. Поскольку $\mathbf{P} \subset \mathbf{coNP}$, нмеем $\mathrm{P} = \mathrm{coNP}$, а значит и $\mathrm{NP} = \mathrm{coNP}$ в предположении $\mathrm{P} = \mathrm{NP}$.

В частности, если $\mathbf{P} = \mathbf{NP}$, то **NP** всё-таки замкнут относительно дополнения. Об истинности обратной теоремы ничего не известно: из $\mathrm{NP} = \mathrm{coNP}$ автоматически не следует $\mathbf{P} = \mathbf{NP}$. В частности, для некоторого оракула A выполнено $\mathrm{NP}^A = \mathrm{coNP}^A$, но $\mathrm{P}^\lambda \neq \mathrm{NP}^\Lambda$.

Для недетерминированных вычислений также можно определять высшие классы, например $\mathrm{NE} = \mathrm{NTIME}(2^{O(\mathrm{n})})$ и $\mathrm{NEXP} = \mathrm{NTIME}(2^{\mathrm{poly(n)}})$. Вопрос $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$ можно перемасштабировать в вощрос $\mathrm{EXP} \stackrel{?}{=} \mathrm{NEXP}$. Между ответами можно установить следующую связь.

Теорема. Если $\mathbf{P} = \mathbf{NP}$, то $\mathbf{EXP} = \mathbf{NEXP}$.

Доказательство. Поскольку $\mathrm{EXP} \subset \mathrm{NEXP}$, достаточно доказать $\mathrm{NEXP} \subset \mathbf{EXP}$. В предположении $\mathbf{NP} \subset \mathbf{P}$ это доказывается при помощи метода єраздутия> (padding argument). Интуитивно, экспоненциальное время работы можно превратить в полиномиальное, если добавить в аргумент экспоненциальное количество ничего не значащего «мусора».

Формально, пусть $A \in \mathbf{NEXP}$, причём A распознаётся некоторой недетерминированной машиной M за время 2^{n^c}. Тогда язык $\tilde{A} = \left\{x01^{2^{n^c}} | n = |x|, x \in A\right\}$ лежит в NP. Действительно, можно сначала легко проверить, что аргумент имеет вид $x01^{20^2}$, а затем применить к x машину M. Время работы машины M составит 2^{n^c}, что будет полиномом (и даже линейной функцией) от длины аргумента $\left|x01^{2^{n^c}}\right| = n + 1 + 2^{n^c}$. В предположении $\mathbf{NP} \subset \mathbf{P}$ найдётся детерминированная машина M', распознающая $\bar{A}$ за время poly $\left(n + 2^{n^c}\right) = 2^{O(n^c)}$. Тогда детерминированная машина, приписывающая к слову x единицы в количестве 2^{n^c} и запускающая M' на полученном слове, будет распознавать A за время $2^{O(n^c)}$, т.е. за экспоненциальное от длины входа время. Значит, $A \in \mathrm{EXP}$, что и требовалось.
Про истинность обратного утверждения (если $\mathbf{EXP} = \mathrm{NEXP}$, то $\mathbf{P} = \mathrm{NP}$) ничего не известно. В частности, существует оракул A, для которого $\mathbf{EXP}^A = \mathrm{NEXP}^A$, но $\mathbf{P}^A \neq \mathbf{NP}^A$.

1.2. Методы эмпирической оценки сложности алгоритмов

Мы разрабатываем и реализуем алгоритмы, создавая иерархию абстрактных операций. которые помогают понять природу решаемых вычислительных проблем. При теоретическом изучении данный процесс, хотя он достаточно важен, может увести очень далеко от проблем реального мира. Поэтому все рассматриваемые нами алгоритмы целесообразно рассматривать в контексте реального языка программирования, например C++. Такой подход иногда оставляет очень туманное

различие между алгоритмом и его реализацией, но это лишь небольшая плата за возможность работать с конкретной реализацией и учиться на ней.

Несомненно, правильно разработанные программы на реальном языке представляют собой эффективные методы выражения алгоритмов. В этой книге мы изучаем большое количество важных и эффективных алгоритмов, кратко и точно реализованных на C++. Описания на обычном языке или абстрактные высокоуровневые представления зачастую неопределённы и незакончены. а реальные реализации заставляют нас находить экономные представления, не позволяющие завязнуть в деталях.

Если алгоритм реализуется как часть большой системы, мы используем абстрактные типы данных или похожий механизм, чтобы иметь возможность изменить алгоритмы или детали реализации после того, как станет ясно, какая часть системы заслуживает наиболее пристального внимания. Однако в самом начале потребуется понимание характеристик производительности каждого алгоритма, так как требования системы к проектированию могут в значительной степени повлиять на производительность алгоритма. Подобные исходные решения при разработке необходимо принимать с осторожностью, поскольку в конце часто оказывается, что производительность целой системы зависит от производительности некоторых базовых алгоритмов.

Алгоритмы нашли эффективное применение во всем многообразии больших программ, операционных систем и различных приложениях. Наше намерение состоит в том. чтобы описать алгоритмы и уделить внимание их динамическим свойствам экспериментальным путем. Для одних приложений приведенные реализации могут подойти точно, для других может потребоваться некоторая доработка. Например, при создании реальных систем требуется более защищенный стиль программирования. Необходимо следить за состоянием ошибок и сообщать о нем, а, кроме того, программы должны быть разработаны таким образом, чтобы их изменение было несложным делом, чтобы их могли быстро читать и понимать другие программисты, чтобы они хорошо взаимодействовали с различными частями системы и сохранялась возможность их переноса в другие среды.

Не отвергая все эти комментарии, все же при анализе каждого алгоритма мы уделяем внимание, в основном, существенным характеристикам производительности алгоритма. Предполагается, что главный интерес будут вызывать наиболее простые алгоритмы с наилучшими показателями производительности.

Для эффективного использования алгоритмов, если нашей целью является решение огромной задачи, которую нельзя решить другим способом, или если цель заключается в эффективной реализации важной части системы, нам требуется понимание характеристик производительности алгоритмов. Формирование такого понимания и является целью анализа алгоритмов.

Один из первых шагов для продвижения вперед в понимании производительности алгоритмов – это эмпирический анализ. Если есть два алгоритма

для решения одной задачи, то в методе нет никакого волшебства: мы запустим оба и определим, который выполняется дольше! Это концепция может показаться слишком очевидной, чтобы о ней стоило говорить, но, тем не менее, это распространенное упущение в сравнительном анализе алгоритмов. Тот факт, что один алгоритм в 10 раз быстрее другого, вряд ли ускользнет от взора того, кто ждет 3 секунды при выполнении одного и 30 секунд – при выполнении другого, однако его легко упустить как небольшой постоянный фактор при математическом анализе. Когда мы отслеживаем производительность точных реализаций алгоритмов при типовом вводе, мы получаем результаты, которые не только являются прямым индикатором эффективности, но и содержат информацию, необходимую для сравнения алгоритмов и обоснования прилагаемых математических результатов. Когда эмпирическое изучение начинает поглощать значительное количество времени, на помощь приходит математический анализ. Ожидание завершения программы в течение часа или целого дня вряд ли может рассматриваться как продуктивный способ для понимания того, что программа медлительна, особенно в тех случаях, когда прямой анализ может привести к аналогичному результату.

Первая проблема, возникающая перед нами в эмпирическом анализе – это разработка корректной и полной реализации. Для некоторых сложных алгоритмов эта проблема может оказаться сложным препятствием. В соответствии с этим, обычно требуется либо с помощью анализа, либо путем экспериментирования с похожими программами, установить некоторый признак, насколько эффективной может быть программа, до того как тратить усилия на ее реализацию.

Вторая проблема, с которой мы сталкиваемся при эмпирическом анализе, – это определение природы входных данных и других факторов, оказывающих прямое влияние на производимые эксперименты. Обычно существуют три основных возможности: реальные данные, случайные данные и ошибочные данные. Реальные данные позволяют точно измерить параметры используемой программы; случайные данные гарантируют, что наши эксперименты тестируют алгоритм, а не данные; ошибочные данные гарантируют, что наши программы могут воспринимать любой направленный им ввод. Например, при тестировании алгоритмов сортировки мы запускаем их для работы с данными в виде слов, в виде сгенерированных случайным образом целых чисел и в виде файлов, содержащих одинаковые числа. Проблема определения, какие данные необходимо использовать для сравнения алгоритмов, возникает также и при анализе алгоритмов.

При сравнении различных реализаций легко допустить ошибки, особенно, если в игру вступают разные машины, компиляторы или системы, либо гигантские программы с плохо охарактеризованным вводом. Принципиальная опасность при эмпирическом сравнении программ таится в том, что код для одной реализации может быть создан более тщательно, чем для другой. Изобретателю предлагаемого нового алгоритма необходимо обратить внимание на все аспекты его реализации, чтобы не расходовать слишком много усилий на создание классического

конкурирующего алгоритма. Чтобы быть уверенным в точности эмпирического сравнения алгоритмов, мы должны быть уверены, что каждой реализации уделялось одинаковое внимание.

Один из подходов заключается в построении алгоритмов за счет внесения небольших изменений в алгоритмы, существующие для данной задачи, поэтому сравнительное изучение просто необходимо. В более общем смысле, мы стараемся определить важнейшие абстрактные операции и начинаем сравнивать алгоритмы на основе того, как они используют такие операции. Например, эмпирические результаты позволяют сравнивать языки программирования и среды, поскольку в них используются одинаковые программы, оперирующие одним и тем же набором базовых операций. Для реальной среды программирования эти числа можно превратить в реальные времена выполнения. Чаще всего просто требуется узнать, какая из двух программ будет быстрее или до какой степени определенное изменение может улучшить временные либо пространственные характеристики программы.

Возможно, наиболее распространенной ошибкой при выборе алгоритма является упущение характеристик производительности. Более скоростные алгоритмы, как правило, сложнее, чем прямые решения, и разработчики часто предпочитают более медленные алгоритмы, дабы избежать дополнительных сложностей. Однако, как это было для алгоритмов union-find, можно добиться значительных улучшений с помощью даже нескольких строк кода. Пользователи удивительно большого числа компьютерных систем теряют существенное время, ожидая, пока простые квадратичные алгоритмы решат задачу, в то время как доступные $N\log N$ или линейные алгоритмы ненамного сложнее, но могут решить задачу быстрее. Когда мы имеем дело с большими задачами, у нас нет другого выбора, кроме как искать наилучший алгоритм, что и будет показано далее.

Возможно, вторая наиболее распространенная ошибка при выборе алгоритма, – уделять слишком много внимания характеристикам его производительности. Снижение времени выполнения программы в 10 раз несущественно, если ее выполнение занимает несколько микросекунд. Даже, если программа требует нескольких минут, время и усилия, необходимые, чтобы она стала выполняться в 10 раз быстрее, могут не стоить того, в особенности, если программа должна применяться лишь несколько раз. Суммарное время, требуемое для реализации и отладки улучшенного алгоритма, может быть значительно выше, чем время, необходимое для выполнения более медленной программы, – в конце концов, часть работы вполне можно доверить компьютеру. Хуже того, можно потратить значительное количество времени и усилий, применяя идеи, которые должны были бы улучшить программу, но, фактически, не делают этого.

Мы не можем провести эмпирические тесты для программы, которая еще не написана, но можем проанализировать свойства программы и оценить потенциальную эффективность предлагаемого улучшения. Не все предполагаемые

усовершенствования дают реальный выигрыш в производительности, поэтому нужно понять степень улучшений на каждом шаге. Более того, в реализации алгоритма могут быть включены параметры, а анализ поможет нам их правильно установить. Наиболее важно то, что понимая фундаментальные свойства программ и базовую природу использования ими ресурсов, мы имеем потенциальную возможность рассчитать их эффективность на компьютерах, которые еще не созданы, или сравнить их с новыми алгоритмами, которые еще не написаны.

Анализ производительности алгоритма – важная часть его разработки, и одним из способов такой оценки выступает анализ сложности алгоритма [34].

Теория сложности – это изучение того, насколько сложны алгоритмы. Чтобы быть полезным, алгоритму необходимо обладать тремя ключевыми функциями:

1. Он должен быть верным. От алгоритма мало пользы, если он нс дает правильных ответов.

2. Хороший алгоритм должен быть понятным для компьютера. Лучший алгоритм в мире окажется бесполезным, если его слишком сложно реализовать.

3. Хороший алгоритм должен быть эффективным. Невозможно использовать алгоритм, который даст правильный результат, но при этом на его работу уйдет тысяча лет или потребуется 1 миллиард терабайт памяти.

Существуют два типа анализа для количественной оценки сложности алгоритма:

1. Анализ пространственной сложности (space complexity analysis) – оценка требований к памяти во время выполнения алгоритма.

2. Анализ временной сложности (time complexity analysis) – оценка времени, необходимого для выполнения алгоритма.

При анализе пространственной сложности оценивают объем памяти, необходимый алгоритму для хранения структур временных данных в процессе работы. Способ разработки алгоритма влияет па количество, тип и размер этих структур данных. В эпоху распределенных вычислений и постоянно растущих объемов данных, которые необходимо обрабатывать, анализ пространственной сложности приобретает все большее значение. Размер, тип и количество структур данных определяют требования к памяти для соответствующего оборудования. Современные структуры данных, используемые в распределенных вычислениях, такие как устойчивые распределенные наборы данных (Resilient Distributed Datasets, RDDs), должны иметь эффективные механизмы распределения ресурсов, учитывающие требования к памяти на различных этанах выполнения алгоритма.

Анализ пространственной сложности необходим для эффективного проектирования алгоритмов. Если при разработке алгоритма не провести надлежащий анализ, то недостаток памяти для временных структур данных может привести к ненужным перегрузкам диска. Это способно значительно повлиять на производительность и эффективность алгоритма.

Анализ временной сложности позволяет узнать, сколько времени потребуется алгоритму для выполнения задачи, исходя из его структуры. В отличие от

пространственной сложности, временная сложность не зависит от оборудования, на котором будет выполняться алгоритм. Она зависит исключительно от структуры алгоритма. Основная цель анализа временной сложности – ответить на ключевые вопросы: будет ли этот алгоритм масштабироваться? Насколько хорошо алгоритм будет обрабатывать большие наборы данных?

Для этого нужно определить влияние увеличения объема данных па производительность алгоритма и убедиться, что алгоритм точен и хорошо масштабируется. Производительность алгоритма становится все более важным показателем в современном мире «больших данных».

Часто мы можем разработать несколько алгоритмов для решения одной и той же задачи. В таком случае нужно проанализировать временную сложность, чтобы ответить на следующий вопрос: «С учетом обозначенной проблемы какой из нескольких алгоритмов наиболее эффективен с точки зрения экономии времени?»

Существуют два основных подхода к вычислению временной сложности алгоритма:

1. Профилирование после реализации. При данном подходе реализуются различные алгоритмы-кандидаты и сравнивается их производительность.

2. Теоретический подход до реализации. При этом подходе производительность каждого алгоритма математически аппроксимируется перед запуском алгоритма.

Преимущество теоретического подхода заключается в том, что он зависит лишь от структуры самого алгоритма. Он не зависит от оборудования, на котором будет выполняться выбранный алгоритм, или от языка программирования, используемого для реализации алгоритма.

Производительность алгоритма обычно зависит от типа входных данных. Например, если данные уже отсортированы в соответствии с контекстом задачи, алгоритм может работать невероятно быстро. Если отсортированные входные данные используются для проверки конкретного алгоритма, то он даст неоправданно высокое значение производительности. Это не будет истинным отражением настоящей производительности алгоритма в большинстве сценариев. Чтобы решить проблему зависимости алгоритмов от входных данных, мы должны учитывать различные типы сценариев при проведении анализа производительности.

При *наилучшем сценарии* входные данные организованы таким образом, чтобы алгоритм обеспечивал наилучшую производительность. Анализ наилучшего сценария дает верхнюю границу производительности.

Второй способ оценить производительность алгоритма – попытаться найти наихудший сценарий, т.е. максимально возможное время, необходимое для выполнения задачи при заданном наборе условий. Такой анализ алгоритма весьма полезен, поскольку мы при этом гарантируем, что независимо от условий производительность алгоритма всегда будет лучше результатов нашего анализа. Анализ наихудшего сценария лучше всего подходит для оценки производительности

при решении сложных проблем с большими наборами данных. Анализ наихудшего сценария даст нижнюю границу производительности алгоритма.

Средний сценарий начинается с разделения всех возможных входных данных па группы. Затем проводится анализ производительности на основе ввода данных от каждой группы. Далее вычисляется среднее значение производительности для каждой из групп.

Анализ среднего сценария не всегда точен, так как он должен учитывать все возможные комбинации входных данных, что не всегда легко сделать.

Эффективность работы алгоритма можно установить, используя теоретический либо экспериментальный методы его исследования [36]. Теоретический метод предполагает использование специальных методов математического анализа алгоритмов, целью которого является установление свойств функции сложности изучаемого алгоритма. При использовании экспериментального метода свойства функции сложности изучаются путем проведения соответствующих расчетов, которые выполняются с использованием специальных методик.

При эмпирическом исследовании используется два метода:

1. измерение временных промежутков;

2. подсчет числа критических операций или групп операций.

У каждого из указанных двух подходов есть свои плюсы и минусы. Математический анализ позволяет получить результаты, которые носят общий, фундаментальный и строгий характер. В то же время сам процесс математического анализа заданного алгоритма может быть весьма сложным, он требует непростой техники построения математических доказательств и зачастую на этом пути просто не удается получить требуемый результат. Последнее сужает круг алгоритмов, которые могут быть изучены таким способом.

Экспериментальный подход применим к намного более широкому кругу алгоритмов, однако получаемые результаты не имеют такого уровня общности и строгости, как в случае математического анализа. В качестве предварительного исследования алгоритма экспериментальный подход вполне уместен и полезен.

Для измерения времени могут быть использованы 2 программных таймера: простой (время измеряется в секундах) и микросекундный (время измеряется в микросекундах). Простой таймер представлен следующими функциями:

runtimer() – выполняет инициализацию таймера, при этом его значение устанавливается в ноль;

timer() – возвращает текущее время, при этом возвращаемое значение имеет вещественный тип double (секунды).

Микросекундный таймер представлен такими функциями:

runstimer() – выполняет инициализацию также, как и функция runtimer;

mcstimer() – возвращает текущее время, при этом возвращаемое значение имеет целый тип (микросекунды).

Микросекундный таймер позволяет осуществлять более точные измерения временных промежутков. Разрешающая способность простого таймера составляет 0,05 с, в то время как разрешение микросекундного таймера составляет 1 мкс.

Если измерение времени для данного алгоритма оказывается невозможным или не продуктивным, в процессе эмпирического исследования зачастую можно использовать метод прямого подсчета количества критических операций (или групп операций).

Для этого следует выполнить предварительный анализ, в результате которого выявить т.н. критические операции (группы операций). Критическими операциями называют те операции, которые выполняются наиболее часто и время выполнения которых составляет основную часть общего времени выполнения алгоритма.

После того, как критические операции определены, в программу вносятся операторы, подсчитывающие их число при выполнении алгоритма.

В разработке есть также и другой немаловажный процесс, жизненно необходимый при создании крупных реальных проектов, это – оптимизация алгоритмов [35]. С точки зрения такого разделения, проектирование алгоритма можно рассматривать как способ достижения заданного асимптотического времени работы (с помощью разработки эффективного алгоритма), а оптимизацию – как процесс уменьшения скрытых временных затрат при сохранении асимптотической сложности алгоритма.

Отдельные рекомендации по алгоритмам проектирования трудно спроецировать на отдельные настройки и приемы, которые приведут к лучшей производительности в конкретной задаче, над которой вы работаете, или для вашего оборудования или версии языка программирования. Это именно тот вид особенностей асимптотики, который нужно избегать. А в некоторых случаях такие приемы могут вовсе не потребоваться, потому что ваша программа и так достаточно быстра. Самое простое, что вы можете делать в большинстве случаев, это просто пробовать и смотреть. Если у вас есть идея, то просто опробуйте ее. Реализуйте прием и запустите несколько тестов. Есть ли улучшения? А если он делает ваш код менее читаемым, а прирост производительности мал, то подумайте – а стоит ли оно того?

Практические рекомендации экспериментальной оценки алгоритмов и их реализации.

1. Излишнее беспокойство об асимптотической сложности может быть очень важным. Иногда это разница между решением и его отсутствием. Однако, постоянные составляющие времени исполнения часто совсем не критичны. Попробуйте простую реализацию алгоритма для начала, и убедитесь, что она работает стабильно. Работает – не трогай.

2. Для измерения времени выполнения используйте timeit. timeit – это модуль предназначенный для измерения времени исполнения. Хотя для получения действительно надежных данных вам потребуется выполнить кучу работы, для практических целей timeit вполне сгодится. Например:

```
>>> import timeit
>>> timeit.timeit("x = 2 + 2")
0.034976959228515625
>>> timeit.timeit("x = sum(range(10))")
0.92387008666992188
```

timeit можно использовать прямо из командной строки, например:

```
$ python -m timeit -s"import mymodule as m" "m.myfunction()"
```

Существует одна вещь, с которой вы должны быть осторожны при использовании timeit: побочные эффекты, которые будут влиять на повторное исполнение timeit. Функция timeit будет запускать ваш код несколько раз, чтобы увеличить точность, и если прошлые запуски влияют на последующие, то вы окажетесь в затруднительном положении. Например, если вы измеряете скорость выполнения чего-то типа mylist.sort(), список будет отсортирован только в первый раз. Во время остальных тысяч запусков список уже будет отсортированным и это даст нереально маленький результат.

3. Чтобы найти узкие места, используйте профайлер. Часто, для того чтобы понять, какие части программы требуют оптимизации, делаются разные догадки и предположения. Такие предположения нередко оказываются ошибочными. Вместо того, чтобы гадать, воспользуйтесь профайлером. В стандартной поставке Python идет несколько профайлеров, но рекомендуется использовать cProfile. Им так же легко пользоваться, как timeit, но он дает больше подробной информации о том, на что тратится время при выполнении программы. Если основная функция вашей программы называется main, вы можете использовать профайлер следующим образом:

```
import cProfile
cProfile.run('main()')
```

Такой код выведет на печать отчет о времени исполнения различных функций программы. Если в вашей системе нет модуля cProfile, используйте profile вместо него. Больше информации об этих модулях можно найти в документации. Если же вам не так интересны детали реализации, а просто нужно оценить поведение алгоритма на конкретном наборе данных, воспользуйтесь модулем trace из стандартной библиотеки. С его помощью можно посчитать, сколько раз будут выполнены то или иное выражение или вызов в программе.

4. Показывайте результаты графически. Визуализация может стать прекрасным способом, чтобы понять что к чему. Для исследования производительности часто применяются графики (например, для оценки связи количества данных и времени исполнения) и коробчатые диаграммы, отображающие распределение временных

затрат при нескольких запусках. Примеры можно увидеть на рис. 57. Отличная библиотека для построения графиков и диаграмм из Python – matplotlib.

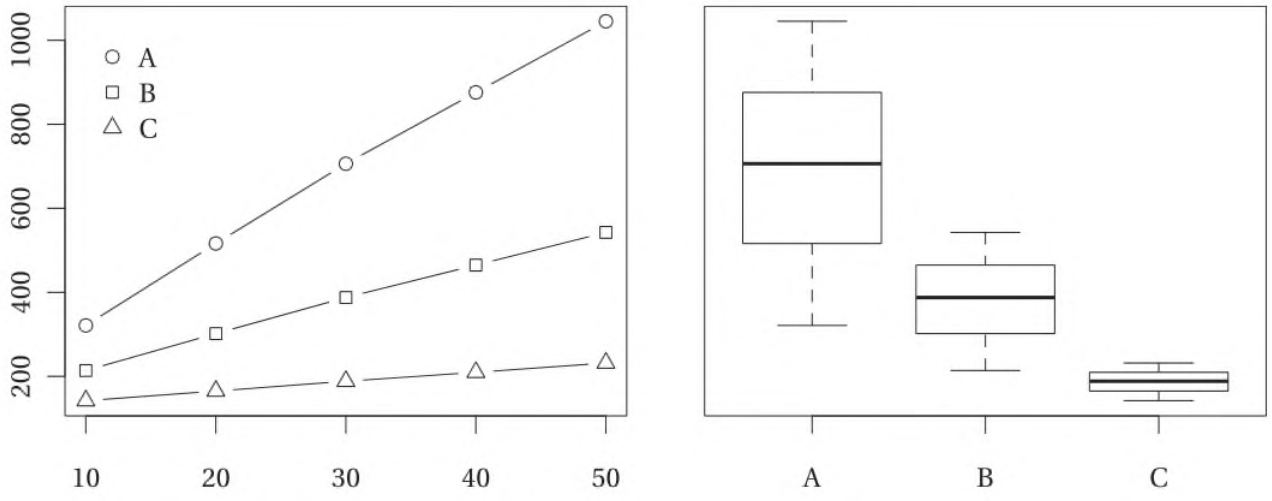


Рисунок 2. Примеры графиков и коробчатых диаграмм

5. Будьте внимательны в выводах, основанных на сравнении времени исполнения. Этот совет довольно расплывчатый, потому что существует много ловушек, в которые можно попасть, делая выводы о том, какой способ лучше, на основании сравнения времени исполнения. Во-первых, любая разница, которую вы видите, может определяться случайностью. Если вы используете специальные инструменты вроде timeit, риск такой ситуации меньше, потому что они повторяют измерение времени вычисления выражения несколько раз (или даже повторяют весь замер несколько раз, выбирая лучший результат). Таким образом, всегда будут случайные погрешности и если разница между двумя реализациями не превышает некоторой погрешности, нельзя сказать, что эти реализации различаются (хотя и то, что они одинаковы, тоже нельзя утверждать).

Проблема усложняется, если вы сравниваете больше двух реализаций. Количество пар для сравнения увеличивается пропорционально квадрату количества сравниваемых версий, сильно увеличивая вероятность того, что как минимум две из версий будут казаться слегка различными. (Это называется проблемой множественного сравнения). Существуют статистические решения этой проблемы, но самый простой способ -- повторить эксперимент с двумя подозрительными версиями. Возможно, потребуется сделать это несколько раз. Они по-прежнему выглядят похожими?

Во-вторых, есть несколько моментов, на которые нужно обращать внимание при сравнении средних величин. Как минимум, вы должны сравнивать средние значения реального времени исполнения. Обычно, чтобы получить показательные числа при измерении производительности, время исполнения каждой программы нормируется делением на время исполнения какого-нибудь стандартного, простого алгоритма. Действительно, это может быть полезным, но в ряде случаев сделает бессмысленными результаты замеров.

И в-третьих, возможно, ваши выводы нельзя обобщать. Подобные измерения на другом наборе входных данных и на другом железе могут дать другие результаты. Если кто-то будет пользоваться результаты ваших измерений, необходимо последовательно задокументировать, каким образом вы их получили.

6. Будьте осторожны, делая выводы об асимптотике из экспериментов. Если вы хотите что-то сказать окончательно об асимптотике алгоритма, то необходимо проанализировать ее, как описано ранее в этой главе. Эксперименты могут дать вам намеки, но они очевидно проводятся на конечных наборах данных, а асимптотика – это то, что происходит при сколь угодно больших размерах данных. С другой стороны, если только вы не работаете в академической сфере, цель асимптотического анализа – сделать какой-то вывод о поведении алгоритма, реализованного конкретным способом и запущенного на определенном наборе данных, а это значит, что измерения должны быть соответствующими.

Предположим, вы подозреваете, что алгоритм работает с квадратичной сложностью времени, но вы не в состоянии окончательно доказать это. Можете ли вы использовать эксперименты для поддержки вашего подозрения? Как уже говорилось, эксперименты (и проектирование алгоритмов) имеют дело в основном с постоянными коэффициентами, но есть способ. Основной проблемой является то, что ваша гипотеза на самом деле не проверяема (через эксперименты). Если вы утверждаете, что алгоритм имеет сложность $O(n^2)$, то данные не могут это ни подтвердить, ни опровергнуть. Тем не менее, если вы сделаете вашу гипотезу более конкретной, то она станет проверяемой. Вы могли бы, например, обоснованно на некоторых данных положить, что время работы программы никогда не будет превышать $0{,}24n^2 + 0{,}1n + 0{,}03$ секунд в вашем окружении. Это проверяемые (или, более точно, опровергаемые) гипотезы. Если вы делаете много измерений, и так и не можете найти контр-примеры, значит ваша гипотеза в какой-то степени верна. Занятная деталь – это, косвенно, подтверждает квадратичную сложность алгоритма.

1.3. Анализ сложности итерационных алгоритмов

В качестве примера определения порядка сложности рассматриваются следующие примеры [37]. Для сокращения объема текста алгоритмы записываются на языке С.

```
for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) {
            a[i][j]++;
      }
}
```

Трудоемкость рассматриваемого фрагмента программы определяется:

двумя операторами начального присвоения,

n операциями инкремента управляющей переменной внешнего цикла,

nm операциями инкремента управляющей переменной внутреннего цикла,

n операциями сравнения во внешнем цикле,

•nm операциями сравнения во внутреннем цикле,

nm операциями нахождения элемента в массиве (вычислений относительного адреса элемента).

nm операциями инкремента.

Суммарная трудоемкость будет определена как сумма всех описанных трудоемкостей, то есть в первом приближении ее можно определить, как $4nm + 2n + 2$ условных операций.

Несмотря на такое значительное число операций сложность алгоритма определяется порядком: $O(nm)$.

При получении порядка сложности рассматриваемого примера учитывалось, что асимптотические обозначения задают класс функций, то есть нет разницы между $O(4nm + 2n + 2)$ и $O(nm)$, поэтому при верхней оценке трудоемкости берется самая быстро растущая функция. В данном случае это $O(nm)$.

```
for (int i = 0; i < n-1; ++i) {
    for (int j = 0; j < n-i; ++j) {
        a[i][j]++;
    }
}
```

В отличие от предыдущего примера, во внутреннем (вложенном) цикле перебираются элементы не до n, а до $n - i$, при этом i не является константой, а зависит от n.

Сложность внутреннего цикла определяется порядком: $O(n - i) = O(n)$. Таким образом, сложность приведенного алгоритма можно оценить как $O(n^2)$.

Понятно, что обработано будет примерно $n^2/2$ элементов матрицы. Из этого примера можно сделать вывод о том, что «игры» с начальным и конечным значением счетчика цикла обычно на трудоемкость не влияют.

```
for (int i = 0; i < n; ++i) {
    for (int j = 1; j < n; j *= 2) {
        a[i][j]++;
    }
}
```

В рассматриваемом примере управляющая переменная внутреннего цикла изменяется в два раза, то есть принимает значения: $1, 2, 4, 8, \ldots 2^k$. При этом $k = \log_2 n$. Таким образом, трудоемкость внутреннего цикла будет оценена как $O(\log n)$, следовательно, трудоемкость всего алгоритма будет равна $O(n \log n)$.

```
for (int i = 0; i < 100; ++i) {
    for (int j = 1; j < n; ++j) {
        ++count;
    }
}
```

Трудоемкость этой задачи определяется порядком $O(100n) = O(n)$.

В данном примере показано, что при асимптотическом анализе можно отбрасывать константы (даже очень большие).

В некоторых задачах Πn может быть ограничено, например: $n < 100000000$. И это не значит, что его всегда можно игнорировать. Очевидно, что будь в рассматриваемом примере такое большое значение – его обязательно нужно считать за n.

```
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n/i; ++j) {
        matrx[i][i]++;
    }
}
```

Во вложенном цикле рассматриваемого примера выполняется n/i итераций.

То есть в i-й строке матрицы matrx обрабатывается n/i элементов.

Трудоемкость обработки i-й строки оценивается порядком:

$$O\left(\sum_{j=0}^{n/i} 1\right) = O\left(\frac{n}{i}\right)$$

Суммарный порядок сложности всего алгоритма будет равен:

$$O\left(\sum_{j=0}^{n} \frac{n}{i}\right) = O\left(n \sum_{j=0}^{n} \frac{1}{i}\right) = O(n \log n).$$

В последней формуле использовано свойство близости в некотором приближении операций суммирования и интегрирования, а также $\int \frac{dx}{x} = \ln x$.

Предыдущий пример и ранее рассмотренный на самом деле сильно отличаются друг от друга. В ранее рассмотренном каждая итерация вложенного цикла выполняла одинаковое число итераций (их количество $\log n$), а в предыдущем для каждой

следующей строки матрицы обрабатывалось меньше столбцов, чем для предыдущей. Тем не менее, асимптотические оценки обоих примеров совпадают.

```
int Factorial(int n){
        int y=l;
        if (n > 1) y = n*Factorial(n-l);
        return y;
}
```

В данном примере функция рекурсивно вызывает саму себя. Сложность рекурсивных алгоритмов зависит не только от сложности внутренних циклов, но и от количества итераций рекурсии. В рассматриваемом примере функция выполняется n раз. Следовательно, вычислительная сложность этого алгоритма равна $O(n)$.

```
void DoubleRecursive(int n) {
        printf(" n = %d\n", n);
        if (n > 0) {
                DoubleRecursive(n-l);
                DoubleRecursive(n-l);
        }
}
```

Рекурсивный алгоритм, который вызывает себя несколько раз, называется многократной рекурсией. Такие функции гораздо сложнее анализировать, кроме того, они существенным образом усложняют алгоритм. Поскольку функция вызывается дважды, можно предположить, что её сложность будет равна $O(2n) = O(n)$. Но на самом деле ситуация гораздо сложнее. При внимательном исследовании этого алгоритма становится очевидным, что его сложность равна $O(2^{n+1} - 1) = O(2^n)$.

Для всех рекурсивных алгоритмов очень важно понятие объёмной сложности. При каждом вызове функция запрашивает небольшой объём памяти для сохранения в стеке значений отдельных переменных (параметров), но этот объём может значительно увеличиваться в процессе рекурсивных вызовов. По этой причине всегда необходимо проводить хотя бы поверхностный анализ объёмной сложности рекурсивных процедур.

Наиболее сложными частями алгоритма обычно является выполнение циклов и обращение к функции. Если одна функция вызывает другую, то необходимо более тщательно оценить сложность последней. Когда в ней выполняется определённое число инструкций (например, вывод на печать), то на оценку сложности это практически не влияет. Если же в вызываемой функции выполняется $O(n)$ элементарных действий (шагов), то эта функция может значительно усложнить

алгоритм. Когда функция вызывается внутри цикла, то влияние может быть намного больше.

В качестве примера рассмотрим две функции: со сложностью $O(n^3)$ и со сложностью $O(n^2)$. Так. если во внутренних циклах первой функции происходит вызов второй функции, то сложности процедур перемножаются. Сложность всего алгоритма составляет $O(n^3)O(n^2) = O(n^5)$. В том случае, когда основная программа вызывает процедуры по очереди, то их сложности складываются: $O(n^3) + O(n^2) = O(n^3)$.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.